leetcode.com/problems/search-insert-position/description/

Problem List | Submit | Premium

Description | Accepted | Editorial | Solutions | Submissions

35. Search Insert Position

Solved

Easy | Topics | Companies

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [1,3,5,6], target = 5
Output: 2
```

Example 2:

```
Input: nums = [1,3,5,6], target = 2
Output: 1
```

18.6K | 425 | 238 Online

Code

Java | Auto

```java
class Solution {
    public int searchInsert(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while(l <= r){
            int m = l + (r - l) / 2;
            if(nums[m] == target) return m;
            else if(nums[m] < target) l = m + 1;
            else r = m - 1;
        }
        return l;
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =

39. Combination Sum

Solved

Medium | Topics | Companies

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all **unique combinations** of* `candidates` *where the chosen numbers sum to* `target`. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than `150` combinations for the given input.

Example 1:

> **Input:** `candidates = [2,3,6,7], target = 7`
> **Output:** `[[2,2,3],[7]]`

20.8K | 223 | 270 Online

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(candidates, target, 0, new ArrayList<>(), res);
        return res;
    }
    void backtrack(int[] arr, int target, int start, List<Integer> temp, List<List<Integer>> res){
        if(target == 0){
```

Saved | Ln 20, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

candidates =

leetcode.com/problems/combination-sum-ii/description/?envType=problem-list-v2&envId=array

Description | Accepted | Editorial | Solutions | Submissions

40. Combination Sum II

Solved

Medium | Topics | Companies

Given a collection of candidate numbers (`candidates`) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sum to `target`.

Each number in `candidates` may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
[1,1,6],
[1,2,5],
```

12.2K | 224 | 144 Online

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> res = new ArrayList<>();
        backtrack(candidates, target, 0, new ArrayList<>(), res);
        return res;
    }
    void backtrack(int[] arr, int target, int start, List<Integer> temp, List<List<Integer>> res){
```

Saved | Ln 21, Col 2

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2

Input

candidates =

45. Jump Game II

Solved

Medium | Topics | Companies

You are given a **0-indexed** array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

- `0 <= j <= nums[i]` and
- `i + j < n`

Return *the minimum number of jumps to reach index* `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

16.4K | 273 | 179 Online

Code

Java | Auto

```java
        for(int i = 0; i < nums.length - 1; i++){
            far = Math.max(far, i + nums[i]);
            if(i == end){
                jumps++;
                end = far;
            }
        }
        return jumps;
    }
}
```

Saved | Ln 13, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =

leetcode.com/problems/group-anagrams/description/?envType=problem-list-v2&envId=array

Array | Submit | Premium

Description | Accepted | Editorial | Solutions | Submissions

49. Group Anagrams

Solved

Medium | Topics | Companies

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

> **Input:** `strs = ["eat","tea","tan","ate","nat","bat"]`
>
> **Output:** `[["bat"],["nat","tan"],["ate","eat","tea"]]`
>
> **Explanation:**
>
> - There is no string in strs that can be rearranged to form `"bat"`.
> - The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.

21.8K | 382 | 422 Online

Code

Java | Auto

```java
        for(String s : strs){
            char[] ch = s.toCharArray();
            Arrays.sort(ch);
            String key = new String(ch);
            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

Saved | Ln 13, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

strs =

66. Plus One

Solved

Easy | Topics | Companies

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the ith digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading `0`'s.

Increment the large integer by one and return *the resulting array of digits.*

Example 1:

```
Input: digits = [1,2,3]
Output: [1,2,4]
Explanation: The array represents the integer 123.
Incrementing by one gives 123 + 1 = 124.
Thus, the result should be [1,2,4].
```

11.7K | 671 | 184 Online

Code

Java | Auto

```
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }
        int[] res = new int[digits.length+1];
        res[0] = 1;
        return res;
    }
}
```

Saved | Ln 14, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

digits =

73. Set Matrix Zeroes

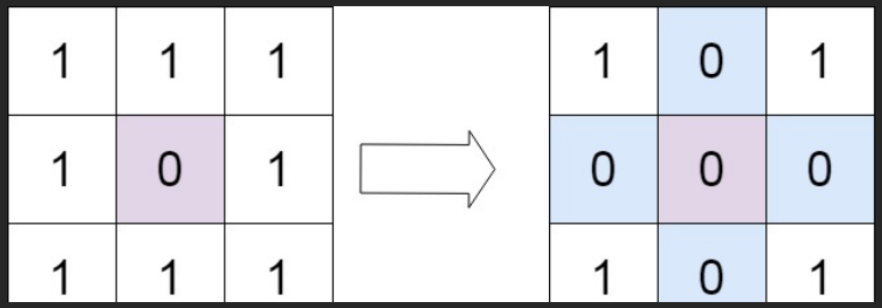
Solved

Medium | Topics | Companies | Hint

Given an `m x n` integer matrix `matrix`, if an element is `0`, set its entire row and column to `0`'s.

You must do it in place.

Example 1:



17K | 323 | 216 Online

Code

Java | Auto

```
        for(int i=1;i<m;i++)
            for(int j=1;j<n;j++)
                if(matrix[i][0]==0 || matrix[0][j]==0)
                    matrix[i][j]=0;

        if(fc) for(int i=0;i<m;i++) matrix[i][0]=0;
        if(fr) for(int j=0;j<n;j++) matrix[0][j]=0;
    }
}
```

Saved | Ln 23, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

matrix =

74. Search a 2D Matrix

Solved

Medium | Topics | Companies

You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if* `target` *is in* `matrix` *or* `false` *otherwise.*

You must write a solution in `O(log(m * n))` time complexity.

Example 1:

17.8K | 350 | 148 Online

Code

Java | Auto

```
        while(l <= r){
            int mid = l + (r-l)/2;
            int val = matrix[mid/n][mid%n];
            if(val == target) return true;
            else if(val < target) l = mid+1;
            else r = mid-1;
        }
        return false;
    }
}
```

Saved | Ln 14, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

matrix =

leetcode.com/problems/sort-colors/description/?envType=problem-list-v2&envId=array

Description | Accepted | Editorial | Solutions | Submissions

75. Sort Colors

Solved

Medium | Topics | Companies | Hint

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

> **Input:** nums = [2,0,2,1,1,0]
> **Output:** [0,0,1,1,2,2]

Example 2:

21.4K | 649 | 248 Online

Code

Java | Auto

```
                l++; m++;
            } else if(nums[m]==1){
                m++;
            } else{
                int t=nums[m]; nums[m]=nums[r]; nums[r]=t;
                r--;
            }
        }
    }
}
```

Saved | Ln 16, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =

leetcode.com/problems/subsets/description/?envType=problem-list-v2&envId=array

Array | Submit | Premium

Description | Accepted | Editorial | Solutions | Submissions

78. Subsets

Solved

Medium | Topics | Companies

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

19.1K | 236 | 198 Online

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums,0,new ArrayList<>(),res);
        return res;
    }
    void backtrack(int[] nums,int start,List<Integer> temp,
List<List<Integer>> res){
        res.add(new ArrayList<>(temp));
        for(int i=start;i<nums.length;i++){
```

Saved | Ln 15, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =

leetcode.com/problems/word-search/?envType=problem-list-v2&envId=array

Array | Submit | Premium

Description | Editorial | Solutions | Submissions | Run Ctrl '

79. Word Search

Medium | Topics | Companies

Given an `m x n` grid of characters `board` and a string `word`, return `true` *if* `word` *exists in the grid.*

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

17.5K | 332 | 165 Online

Code

Java | Auto

```java
class Solution {
    public boolean exist(char[][] board, String word) {
        int m=board.length,n=board[0].length;
        for(int i=0;i<m;i++)
            for(int j=0;j<n;j++)
                if(dfs(board,word,i,j,0)) return true;
        return false;
    }
    boolean dfs(char[][] b,String w,int i,int j,int k){
        if(k==w.length()) return true;
```

Saved | Ln 19, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

board =

leetcode.com/problems/4sum/?envType=problem-list-v2&envId=array

Array | Submit | Premium

Description | Editorial | Solutions | Submissions

18. 4Sum

Medium | Topics | Companies

Given an array `nums` of `n` integers, return *an array of all the* ***unique*** *quadruplets* `[nums[a], nums[b], nums[c], nums[d]]` such that:

- `0 <= a, b, c, d < n`
- `a`, `b`, `c`, and `d` are **distinct**.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

You may return the answer in **any order**.

Example 1:

```
Input: nums = [1,0,-1,0,-2,2], target = 0
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]
```

12.8K | 379 | 171 Online

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        Arrays.sort(nums);
        List<List<Integer>> res=new ArrayList<>();
        int n=nums.length;
        for(int i=0;i<n-3;i++){
            if(i>0&&nums[i]==nums[i-1]) continue;
            for(int j=i+1;j<n-2;j++){
                if(j>i+1&&nums[j]==nums[j-1]) continue;
                int l=j+1,r=n-1;
```

Saved | Ln 25, Col 2

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2

Input

nums =

33. Search in Rotated Sorted Array

Medium | Topics | Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`*, or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

29.7K | 556 | 405 Online

Code

Java | Auto

```java
class Solution {
    public int search(int[] nums, int target) {
        int l=0,r=nums.length-1;
        while(l<=r){
            int m=l+(r-l)/2;
            if(nums[m]==target) return m;
            if(nums[l]<=nums[m]){
                if(target>=nums[l]&&target<nums[m]) r=m-1;
                else l=m+1;
            } else{
```

Saved | Ln 17, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =

leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/?envType=problem-list-v2&envId=array

Description | Editorial | Solutions | Submissions

34. Find First and Last Position of Element in Sorted Array

Medium | Topics | Companies

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
```

Example 2:

23.1K | 354 | 238 Online

Code

Java | Auto

```java
class Solution {
    public int[] searchRange(int[] nums, int target) {
        return new int[]{first(nums,target), last(nums,target)};
    }
    int first(int[] a,int t){
        int l=0,r=a.length-1,res=-1;
        while(l<=r){
            int m=l+(r-l)/2;
            if(a[m]>=t) r=m-1;
            else l=m+1;
```

Saved | Ln 25, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =